Assignment- 14.5

Name :Rishik

Hall Ticket No:2303a51778

Batch : 12

Task 1 – Image Gallery Website

Create an image gallery web page.

Requirements:

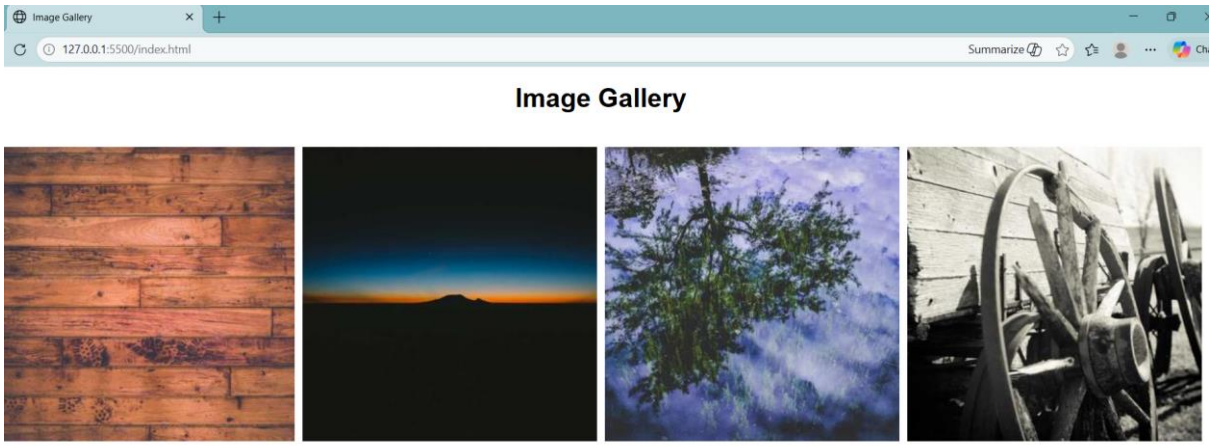
• Display images in a grid layout.

• Use AI to:

o Optimize layout using CSS Grid.

o Add hover zoom or overlay effects.

o Ensure mobile responsiveness.

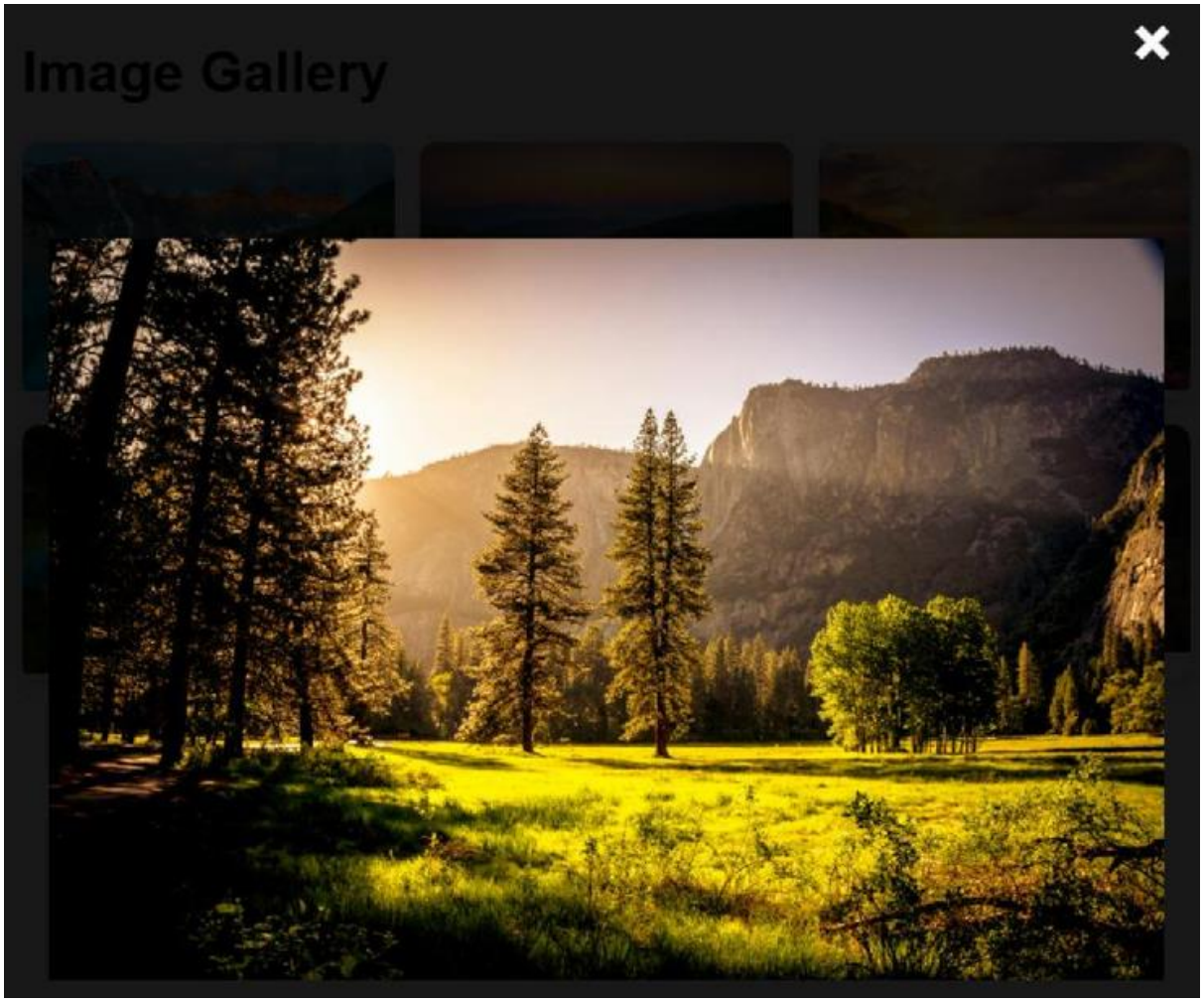


Explanation :

I understood that this task is about creating a visually appealing image gallery using HTML, CSS, and JavaScript, where images are arranged in a structured grid layout using CSS Grid. It also involves adding interactive effects like hover zoom or overlays to enhance user experience, and ensuring the design is responsive so that it works smoothly on different devices like mobiles and desktops.

<u>Prompt 2:</u>

Modify the code so that when a image is clicked it should be shown full screen and when the mouse moves to an image the image should become slightly bigger.



Task 2 – Profile Card Generator

Create a profile card web page.

Requirements:

• Display user image, name, role, and social media links.

• Use AI to:

o Suggest modern UI colors and fonts.

o Design the card using Flexbox.

o Add hover animation and shadow effects

PROMPT:

<!-- Create a responsive profile card using HTML and CSS.

Requirements:

- Display profile image, name, role, and social media icons

- Use Flexbox for layout

- Add hover animation and shadow effects

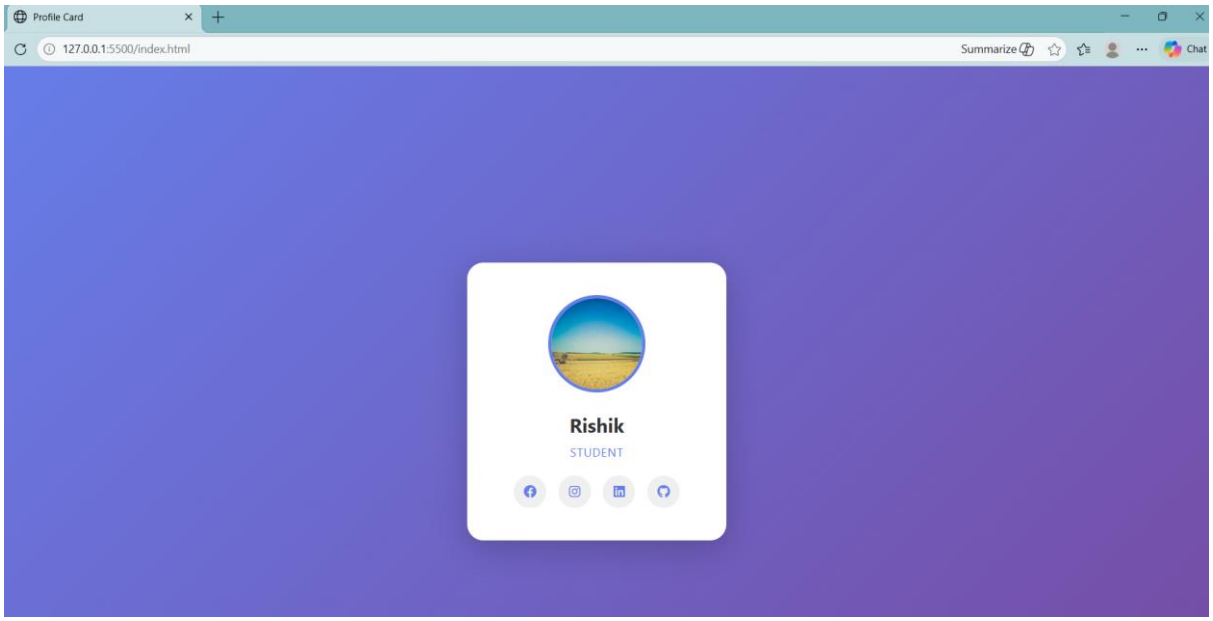
- Use modern colors and fonts

- Center the card on the page

- Make it responsive for mobile devices

Provide complete code (HTML and CSS). -->

OUTPUT:



EXPLANATION:

In this task, I understood how to design a simple and attractive profile card using HTML and CSS. The profile card shows basic user information like profile image, name, role, and social media icons. I learned how to use Flexbox to place the card at the center of the page and align elements properly. I also understood how styling properties like colors, shadows, and border radius make the design look more modern and clean. Overall, this task helped me understand how to create a visually appealing and user-friendly UI component.

Task 3 -Develop a dynamic shopping cart system.

Requirements:

• Add, remove, and update product quantities.

• Display real-time total price.

• Use AI to:

o Manage cart state using JavaScript objects.

o Persist cart data in localStorage.

PROMPT:

<!--Create a dynamic shopping cart system using HTML, CSS, and JavaScript.

Requirements:

- Display a list of products with name, price, and "Add to Cart" button

- Allow users to add items to the cart

- Allow users to increase/decrease quantity and remove items

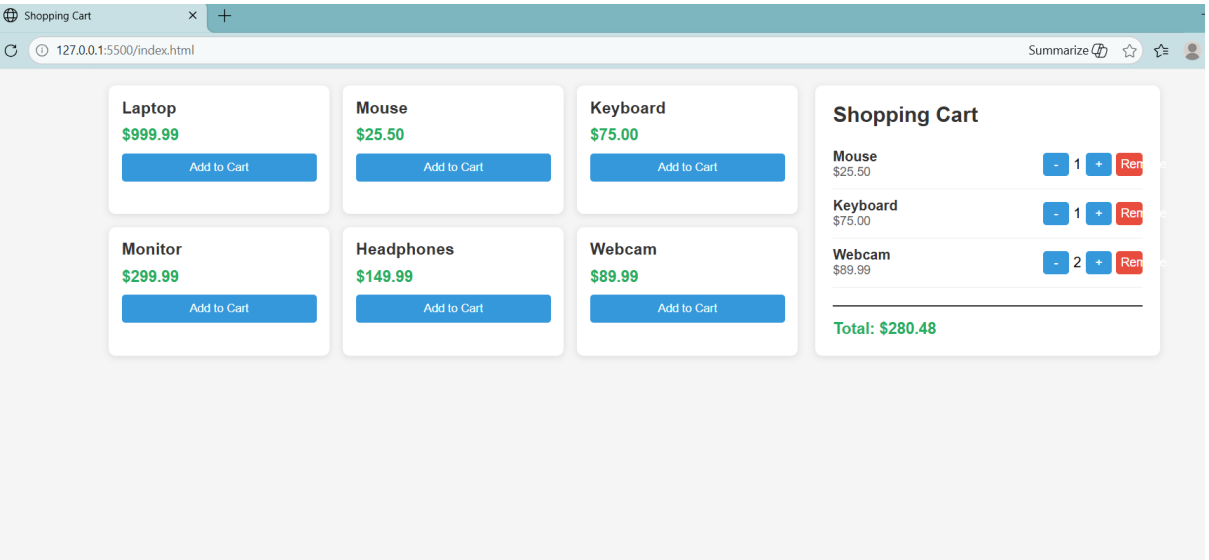
- Show total price dynamically

- Use JavaScript objects/arrays to manage cart data

- Store cart data in localStorage so it persists after refresh

- Make the UI simple, clean, and responsive->>

OUTPUT:



EXPLANATION:

In this task, I understood how to build a dynamic shopping cart system using HTML, CSS, and JavaScript. The application allows users to add products to the cart, update item quantities, and remove items when required. I learned how JavaScript can be used to manage cart data using objects or arrays and update the total price

dynamically whenever changes occur. I also understood the use of localStorage to store cart data so that it persists even after refreshing the page. This task helped me understand how real-time updates and user interactions are handled in web applications, making the system more interactive and user-friendly.

Task 4 – Role-Based Access Control Application

Create a role-based web application.

Requirements:

• Admin, Editor, User roles.

• Conditional UI rendering.

• Use AI to:

o Design access control logic.

o Maintain secure state transitions.

o Create scalable UI components.

PROMPT:

<!--Create a role-based access control web application using HTML, CSS, and JavaScript.

Requirements:

- Define three roles: Admin, Editor, and User

- Show different UI elements based on selected role

  - Admin: full access (view, edit, delete)

  - Editor: can view and edit

  - User: can only view

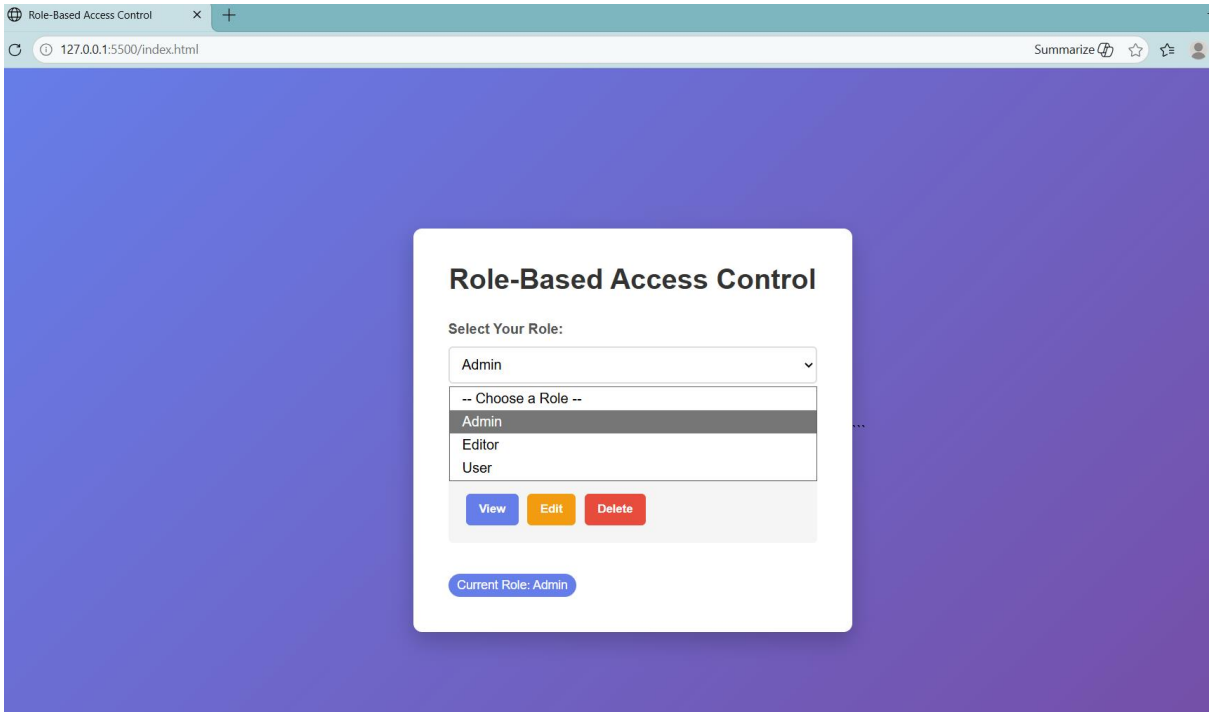
- Add buttons or sections that appear/disappear depending on role

- Use JavaScript to handle role selection and conditional rendering

- Maintain role state using variables or localStorage

- Keep UI simple, clean, and responsive-->

OUTPUT:



EXPLANATION:

In this task, I understood how to create a system where different users like Admin, Editor, and User have different access permissions. Based on the role, some features are shown or hidden using JavaScript. I learned how to control the UI using conditions and manage user roles. This helped me understand how real applications provide access based on user roles.

Task 5 – Multi-Language Website

Design a multilingual website.

Requirements:

• Switch languages dynamically.

• Persist language preference.

• Use AI to:

o Structure translation files.

o Handle RTL/LTR layouts.

o Improve accessibility.

PROMPT:

<!--Create a multi-language website using HTML, CSS, and JavaScript.

Requirements:

- Provide at least two languages (e.g., English and Hindi)

- Add a language switch button or dropdown

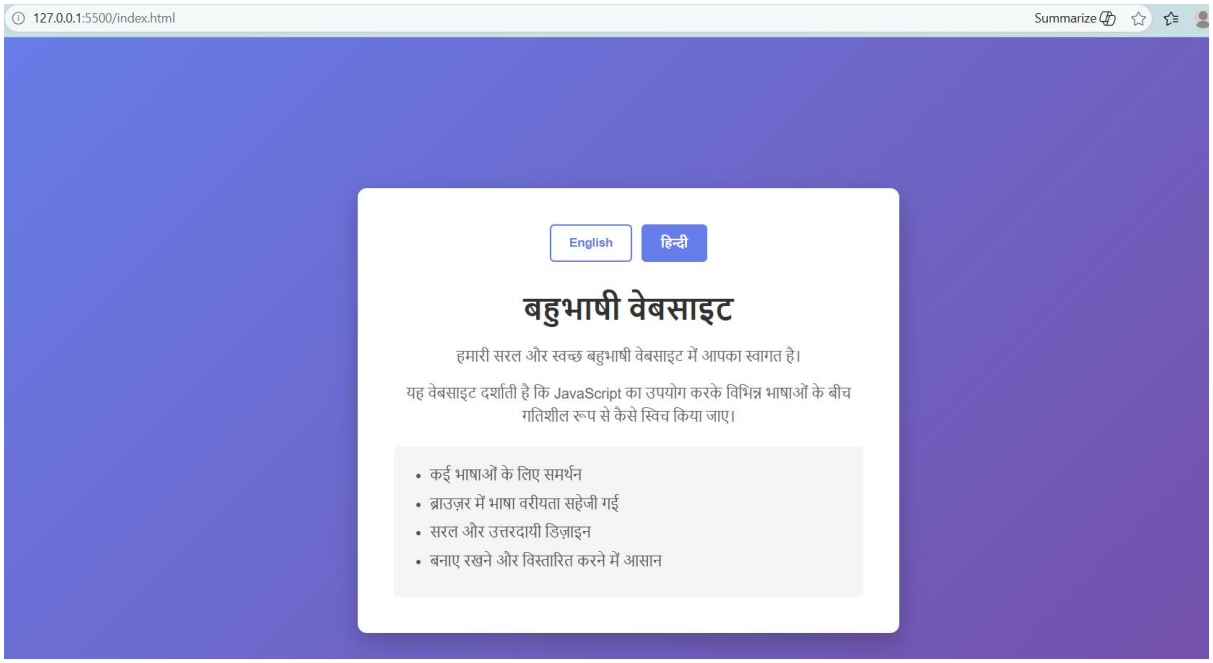
- Change website text dynamically based on selected language

- Store selected language using localStorage

- Use a translation object (JSON format) to manage text

- Ensure the design is simple, clean, and responsive-->

OUTPUT:



EXPLANATION:

In this task, I understood how to create a website that supports multiple languages and allows users to switch between them easily. I learned how to use JavaScript to change the text content dynamically based on the selected language and how to store the selected language using localStorage so it stays even after refreshing. This helped me understand how websites can be made more user-friendly and accessible for people from different regions.

Task 6 – Online Examination System

Create an online exam interface.

Requirements:

• Timer, questions, and submission.

• Prevent multiple submissions.

• Use AI to:

o Manage exam state.

o Improve exam UX.

o Ensure accessibility.

PROMPT:

<!--Create an online examination system using HTML, CSS, and JavaScript.

Requirements:

- Display multiple choice questions with options

- Allow user to select answers

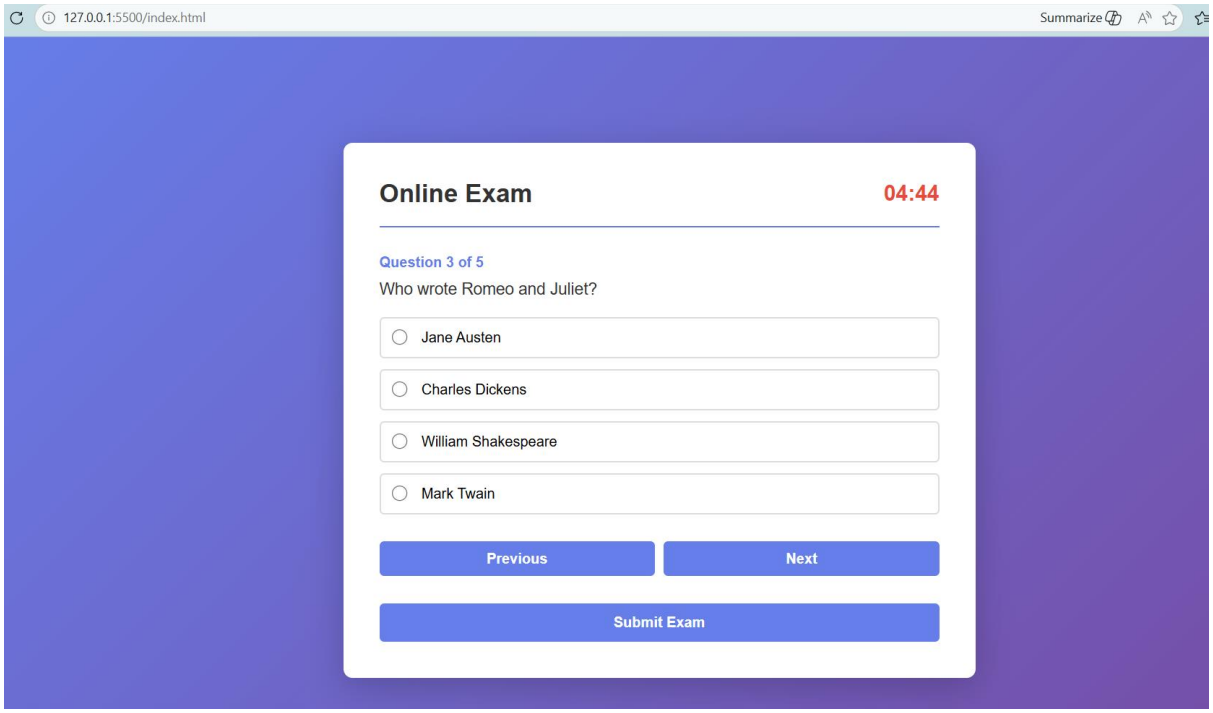
- Add a countdown timer for the exam

- Submit answers and display score at the end

- Prevent multiple submissions after exam is submitted

- Show result immediately after submission-->

OUTPUT:



EXPLANATION:

In this task, I understood how to create an online examination system where users can answer questions within a given time. I learned how to use JavaScript to display

questions, handle user input, and calculate the score after submission. I also understood how to implement a timer and prevent multiple submissions. This task helped me understand how real-time interaction and control flow work in web applications.